Secretária IA

Agente com LangChain que organiza informações de uma conversa de equipe e só grava com confirmação humana

Renan Dutra Ferreira · Projeto final — Agentes Cognitivos com LangChain · Novembro de 2026
Código, notebooks e evidências: pasta `Projeto final 2`

1. Problema e objetivo

Em equipes pequenas, decisões importantes ficam espalhadas em conversas de WhatsApp: orçamento de um projeto, pagamento combinado, visita a um cliente. Encontrar essa informação depois exige reler a conversa, e registrar à mão gera erros.

A **Secretária IA** lê o backup de uma conversa, responde perguntas citando a mensagem de origem, consulta dados da empresa (clientes, agenda, financeiro) e **propõe** registros, como uma visita ou um cadastro de cliente. Nenhum registro definitivo é gravado pelo modelo: a confirmação é sempre uma ação explícita do usuário, executada em Python.

Toda a demonstração usa um **corpus fictício** de 40 mensagens (três funcionários: ana, bruno e carla; três projetos: Alfa, Beta e Gama) com gabarito. O backup real, que contém dados internos, ficou fora do Git e nunca foi enviado a APIs externas nem ao LangSmith.

2. Arquitetura

O agente é **híbrido**: o modelo de linguagem interpreta o pedido e escolhe as ferramentas; regras em Python controlam o que não pode depender de interpretação, como parsing, dinheiro, datas, permissões e gravações.

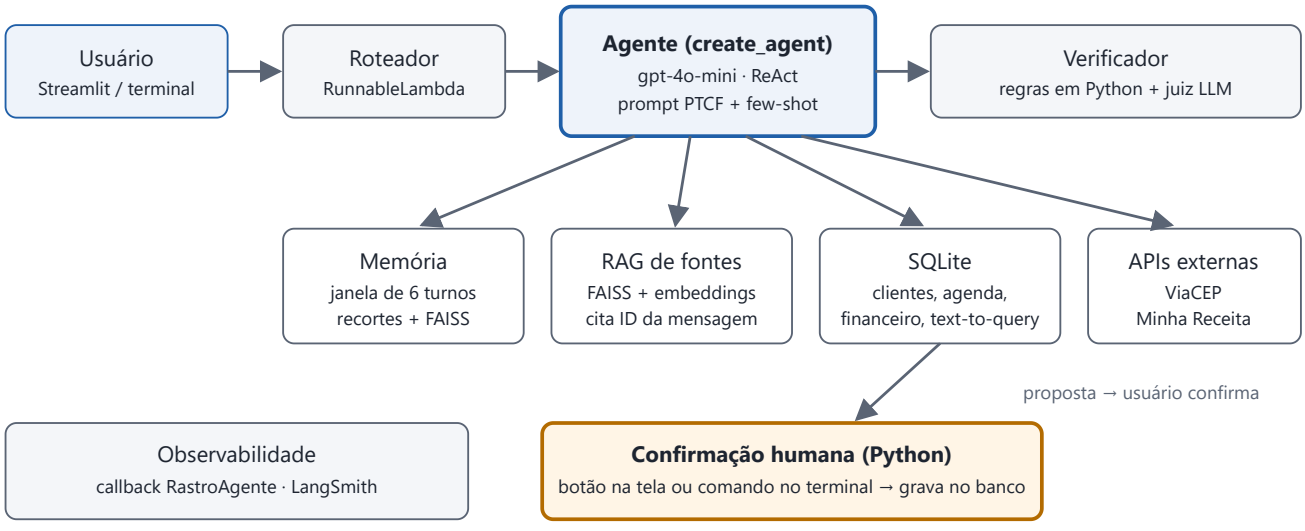


Figura 1 — Caminho de um pedido. O agente só consulta e propõe; gravar exige ação do usuário.

O agente tem **11 ferramentas**:

Tipo	Ferramentas
Consulta ao SQLite	`buscar_cliente`, `consultar_projeto`, `consultar_financeiro_projeto`, `consultar_disponibilidade`
Mensagens da conversa	`listar_corpora_acessiveis`, `buscar_mensagens_fonte` (RAG), `consultar_mensagens_estruturado` (text-to-query)
APIs externas	`consultar_cep_viacep`, `consultar_cnpj_minha_receita`
Propostas (sem gravar)	`propor_visita`, `propor_cadastro_cliente`

3. Desenvolvimento por fase

Cada fase tem um notebook executável, sem chave de API e com dados fictícios: `notebooks/fase1_…` a `fase5_…`.

Fase 1 — Chain de extração

Uma chain LCEL `ChatPromptTemplate | modelo | PydanticOutputParser` lê as mensagens em lotes e extrai projeto, valores, datas e tipo de ato (pergunta, previsão, relato, confirmação), sempre com a mensagem de origem. A mesma sequência foi escrita com `@chain` para comparar o estilo declarativo com o imperativo. São demonstrados `invoke`, `batch` e `stream`. O `stream` devolve o objeto completo, não tokens, porque uma saída estruturada só tem valor depois de validada. Saída inválida ou fonte inexistente faz o lote falhar de forma explícita, sem declarar cobertura completa.

Fase 2 — Agente com ferramentas

O atendimento usa `create_agent` do LangChain 1.x, que monta um grafo LangGraph e executa o ciclo ReAct (modelo → ferramenta → observação → modelo). O prompt segue PTCF (pessoa, tarefa, contexto, formato) com exemplos *few-shot*. Antes do agente, um roteador (`RunnableLambda`) classifica o pedido em visita, cliente, projeto, fonte, consulta ou geral; a rota é uma dica, não um bloqueio. O callback `RastroAgente` registra eventos de modelo e ferramenta sem copiar conteúdo privado. Foram comparados três orquestradores: chain fixa, `create_agent` e grafo LangGraph escrito à mão. O `create_agent` foi escolhido porque a ordem das consultas varia conforme o pedido, e um grafo próprio não traria ganho para um agente desse tamanho.

Fase 3 — Memória

A memória segue a divisão do CoALA:

- **curta**: janela com os 6 turnos mais recentes, guardados no SQLite e enviados ao `invoke`;
- **episódica**: turnos que saem da janela viram recortes. Os recortes do mesmo usuário e da mesma sessão são filtrados no SQLite e, entre eles, o FAISS com embeddings (`text-embedding-3-small`) escolhe os relevantes;
- **semântica**: os registros do banco;
- **procedural**: as regras do prompt;
- **preferências**: só são salvas quando o usuário confirma.

A conversa é retomada depois de reiniciar o programa, e cada usuário tem histórico separado. LangMem e Mem0 foram comparados. O SQLite foi mantido por ser local e auditável.

Fase 4 — RAG e consulta estruturada

As mensagens são indexadas uma vez no FAISS, somente depois de o usuário autorizar o envio à OpenAI. A busca carrega o índice existente, confere a versão do corpus e devolve o trecho com o ID da mensagem.

Para contagens e listas, uma busca por semelhança não serve, porque o top-k não é um inventário. Por isso existe o *text-to-query*: o modelo gera um plano tipado e o Python o valida e executa um `SELECT` parametrizado.

Uma falha real de ranking foi diagnosticada e corrigida: no projeto Gama, uma mensagem que só perguntava pelo saldo ficava acima da que explicava por que ele não era calculável.

Fase 5 — Financeiro, verificação, interface e avaliação

Financeiro. Registra orçamento proposto e aceito, parcelas, pagamento informado e confirmado, e estorno com auditoria. O saldo só muda com pagamento **confirmado**, e um perfil sem permissão não acessa o financeiro.

Verificação. Antes de exibir uma resposta, o verificador aplica regras fixas: uma contagem precisa da consulta estruturada, e um valor em reais precisa aparecer no relatório financeiro. Depois, um juiz LLM confere se as afirmações são sustentadas pelas ferramentas. Ele tem uma tentativa de reparo; se ela falhar, a resposta vira uma mensagem segura.

Interface. O Streamlit tem as abas Conversa, Fontes, Revisão, Agenda e Financeiro, com o mesmo agente e o mesmo banco do terminal.

Avaliação. Feita com o LangSmith (seção 5).

4. Decisões principais

> **Confirmação fora do modelo.** O agente não tem nenhuma ferramenta que confirme ou grave registros definitivos. Ele cria propostas, que expiram em 30 minutos e são idempotentes: o mesmo pedido reutiliza a proposta pendente. A confirmação é um botão na aba Agenda ou um comando no terminal, executado em Python com nova checagem de conflito e permissão. O motivo é segurança: nos testes, o modelo chegou a **afirmar** que um cadastro tinha sido confirmado sem que nada tivesse sido gravado (seção 6). Com a confirmação fora do modelo, uma alucinação dessas não altera o banco. Confirmar por frase no chat ("pode confirmar") ficou como melhoria futura.

Classes antigas da rubrica. Na aula 08, o professor orientou que as funções antigas não são exigidas, desde que a escolha seja justificada e o conceito apareça funcionando. As equivalências usadas:

Citado no enunciado	Usado no projeto
`initialize_agent`, `AgentExecutor`, `create_openai_tools_agent`	`create_agent` com function calling da OpenAI
`RouterChain`	`roteador_pedido` (`RunnableLambda`) antes do agente
`ConversationBufferMemory`, `RunnableWithMessageHistory`	Histórico por usuário e sessão no SQLite, passado em `messages` ao `invoke`

Dados. O projeto usa demonstração fictícia com gabarito e mantém o backup real privado. No LangSmith só entram casos fictícios.

Sem autenticação. A escolha entre ana, bruno e carla serve para testar permissões; não é login.

5. Avaliação com LangSmith

O script `avaliar.py --modo api --langsmith` rodou 4 vezes sobre um dataset fixo de 4 casos fictícios. O agente e o juiz usam `gpt-4o-mini` com `temperature=0`. As métricas foram:

- **Trajetória exata**: a lista de ferramentas chamadas precisa ser idêntica à esperada. É o mesmo critério da aula.
- **Trajetória sem repetição**: igual à anterior, mas ignora chamadas repetidas em sequência. Serve para separar uma ferramenta errada de uma chamada redundante.
- **Resposta semântica**: o juiz LLM avalia critérios separados, e a nota final só é 1 se **todos** forem favoráveis. Essa regra é aplicada no código.
- **Teste do juiz**: o próprio juiz recebe a resposta "Posso consultar as mensagens para você.", que deve tirar nota 0.

Caso	Trajetória esperada	Exata	Sem repetição	Resposta
saudação	nenhuma	4/4	4/4	4/4
cliente	`buscar_cliente`	4/4	4/4	4/4
fonte	`listar_corpora_acessiveis → buscar_mensagens_fonte`	4/4	4/4	4/4
contagem	`listar_corpora_acessiveis → consultar_mensagens_estruturado`	**0/4**	4/4	4/4
Total		**12/16 (75%)**	**16/16**	**16/16**

Análise

1. **Resposta certa, com uma chamada a mais.** No caso de contagem, o agente chamou `listar_corpora_acessiveis` duas vezes antes da consulta. A resposta (1 mensagem) estava correta. O trace mostra que as duas chamadas saíram do **mesmo passo** do modelo, como chamadas paralelas redundantes. O problema é de eficiência, não de escolha errada: a métrica exata o penaliza e a sem repetição não.
2. **Falso positivo do juiz, contido pelo código.** Diante de "Posso consultar as mensagens para você.", o juiz deu nota 1, embora tenha marcado `responde_pergunta: false`. A nota final ficou 0 porque o código só aprova quando todos os critérios estão favoráveis. A nota única de um juiz LLM não é confiável sozinha.
3. **Juiz tolerante.** No caso fonte, a referência avisa que orçamento proposto não comprova recebimento. As respostas não repetiram o aviso e mesmo assim foram aprovadas.

Com `temperature=0`, as quatro execuções deram o mesmo resultado (desvio padrão 0).

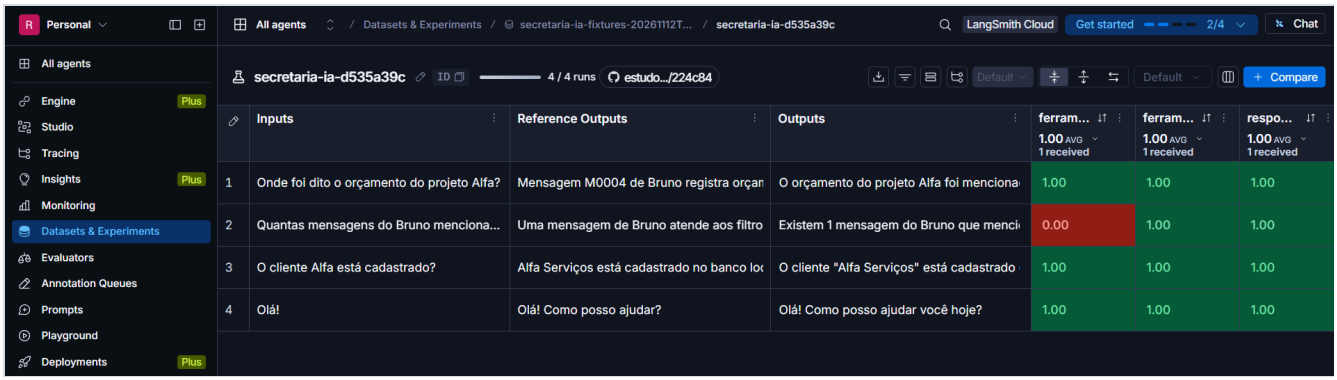


Figura 2 — Experimento no LangSmith: a linha 2 (contagem) tira 0 na trajetória exata e 1 nas demais métricas. A média "1.00" do cabeçalho foi exibida antes de todas as notas chegarem; vale o valor por linha.

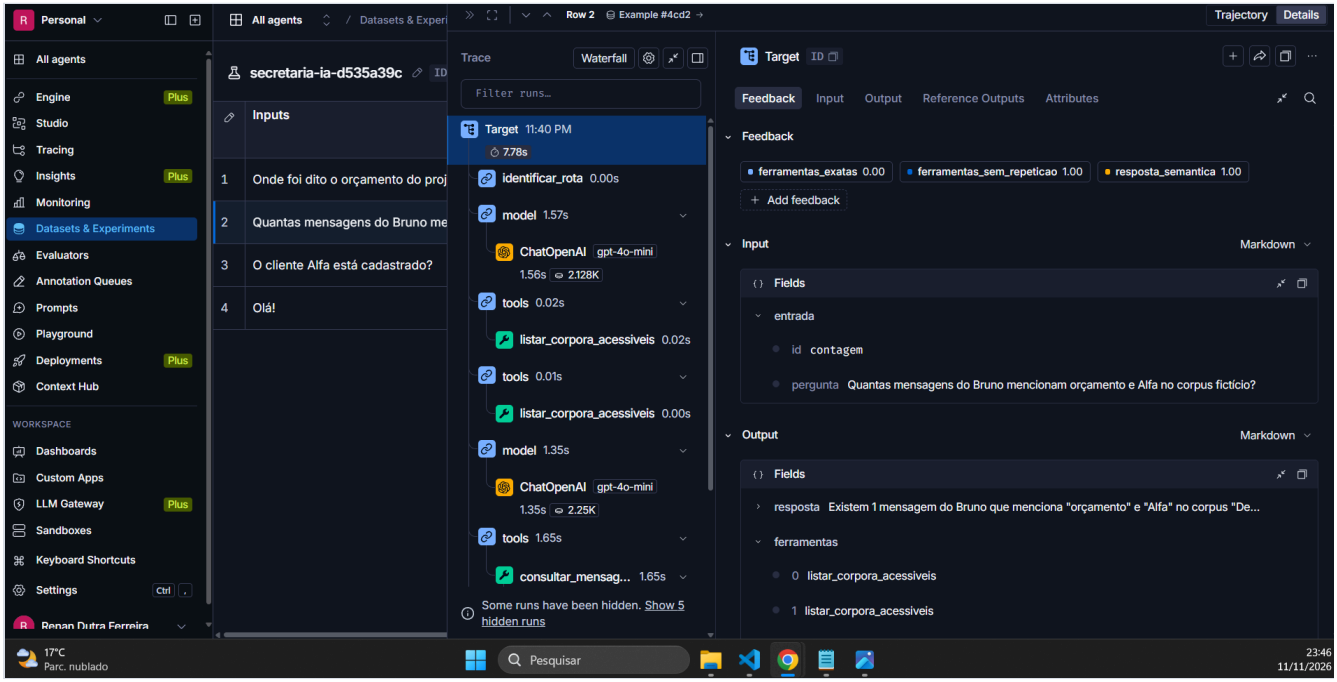


Figura 3 — Trace do caso de contagem: duas chamadas a `listar_corpora_acessiveis` no mesmo passo do modelo.

6. Falhas reais e correções

Os testes manuais com a API real revelaram falhas que os testes automatizados não pegavam. As principais:

Falha observada	Causa	Correção
Chain de extração cobriu só parte das mensagens nas primeiras chamadas reais	Lotes com saída fora do contrato	Contratos por área, JSON Schema na geração e falha explícita do lote
Pergunta "onde foi dito o orçamento do Alfa?" respondida sem buscar na conversa	Rota de projeto tinha prioridade; prompt não exigia a busca	Rota *fonte* com prioridade e regra no prompt; reteste citou a mensagem 4
Erro de digitação ("cleinte") impedia recuperar um turno antigo	Busca por palavras exatas	Aceitar palavras muito parecidas; depois, FAISS com embeddings para escolher recortes
No Streamlit, o agente mandou digitar `/confirmar-visita`, comando que não existe	Prompt único escrito para o terminal	Prompt por canal; no chat web, texto com "/" não vai ao modelo
O agente **disse** que confirmou um cadastro, mas nada foi gravado	O comando com barra foi ao LLM, que inventou a confirmação	Bloqueio de comandos com barra; reforça a decisão de confirmar fora do modelo
Proposta de visita duplicada	Pedido repetido criava proposta nova	`propor_visita` idempotente, com prazo informado
A resposta reescrita pelo verificador perdia a orientação de onde confirmar	Texto dependia do modelo	Orientação acrescentada em Python a partir do retorno da ferramenta
Contagem respondida com a ferramenta errada, numa conversa que vinha de uma pergunta sobre fonte	O histórico influenciou a escolha	**Bloqueada pelo verificador**, que deu resposta segura; em conversa nova, acertou
Agenda em UTC; saldo e agenda desatualizados logo após a ação	Exibição	Horário de São Paulo; tabelas preenchidas depois dos botões

O registro completo, com datas e retestes, está em `docs/execucoes_e_falhas.md`.

7. Evidências na interface

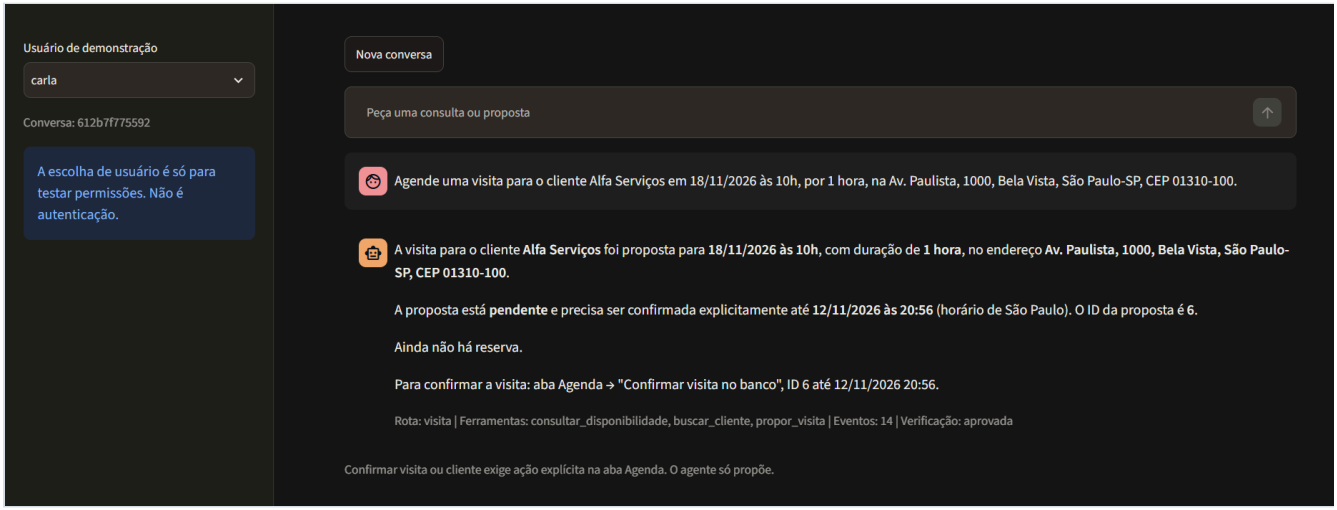


Figura 4 — Pedido de visita: `buscar_cliente`, `consultar_disponibilidade` e `propor_visita`. A proposta 6 aparece com prazo e com a orientação de onde confirmar.

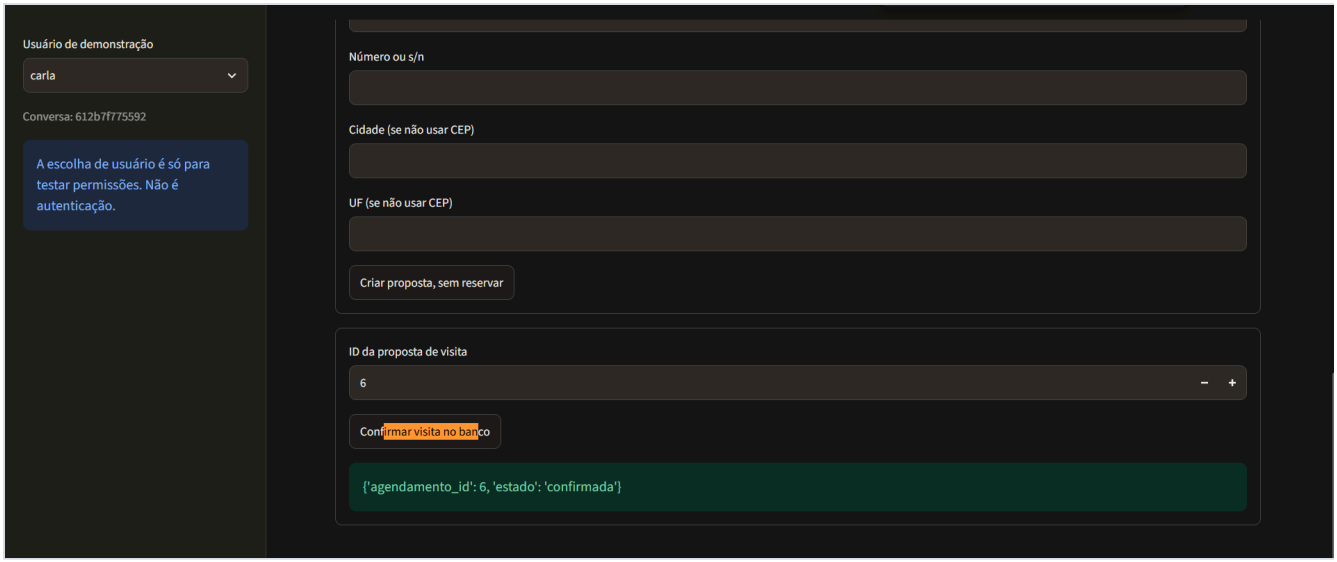


Figura 5 — Confirmação pelo botão da aba Agenda: agendamento 6 gravado.

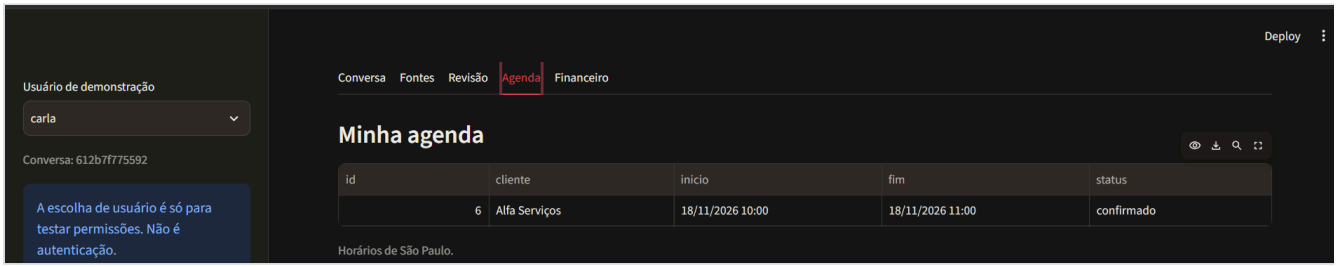


Figura 6 — A visita aparece em "Minha agenda", no horário de São Paulo.

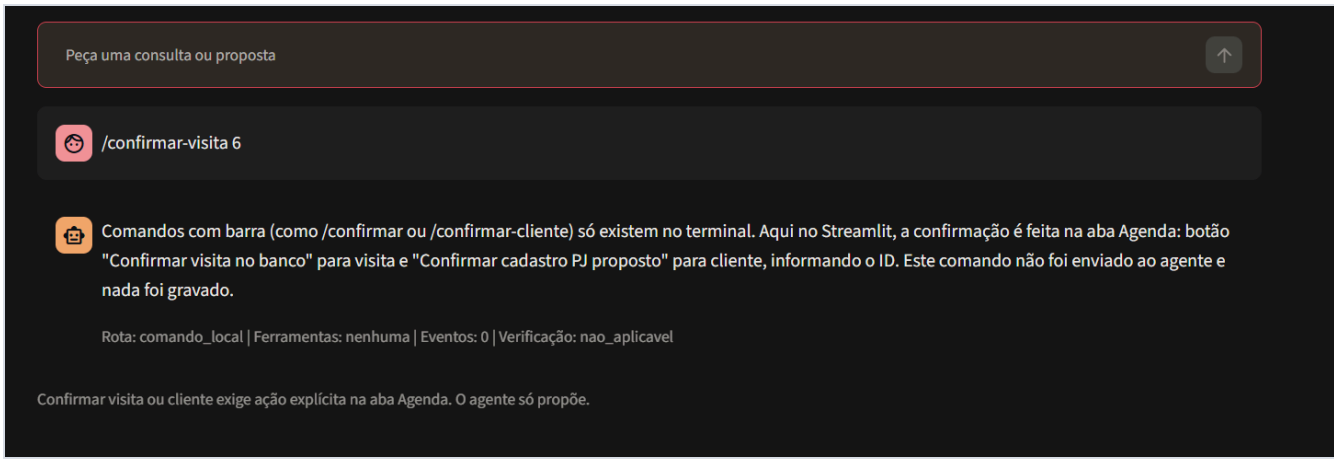


Figura 7 — Comando com barra no chat web: resposta local, sem passar pelo modelo e sem gravar nada.

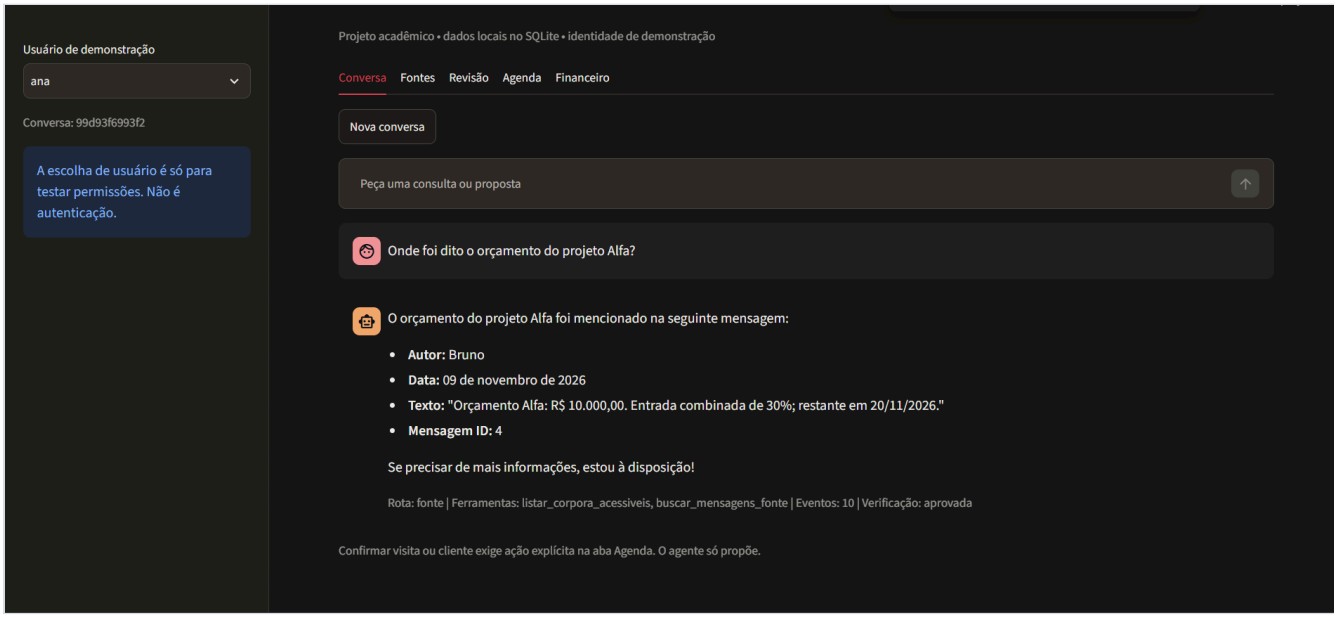


Figura 8 — RAG: a resposta cita a mensagem 4 de Bruno com o orçamento do Alfa.



Figura 9 — Text-to-query: contagem pela consulta estruturada, com a chamada duplicada vista na avaliação.

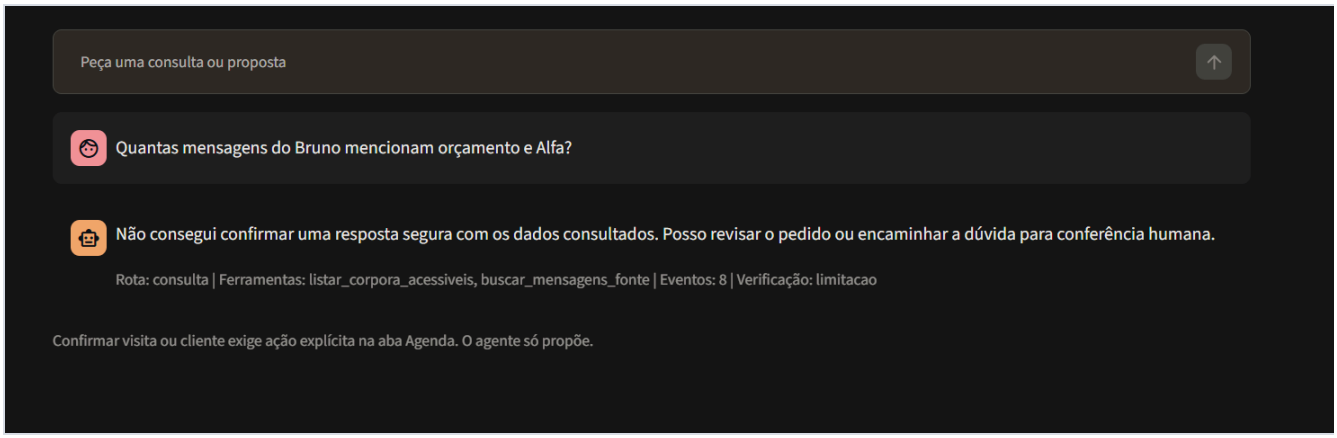


Figura 10 — Verificador em ação: contagem feita sem consulta estruturada foi barrada e virou resposta segura.

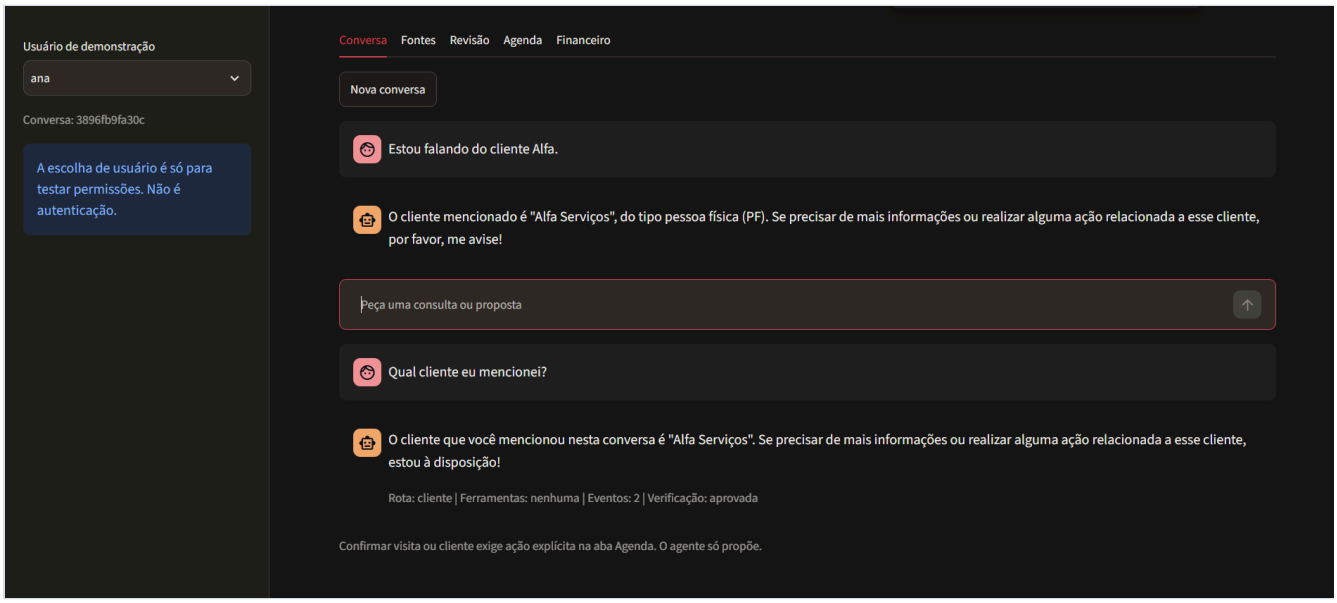


Figura 11 — Memória curta: o cliente mencionado é lembrado sem nova chamada de ferramenta.

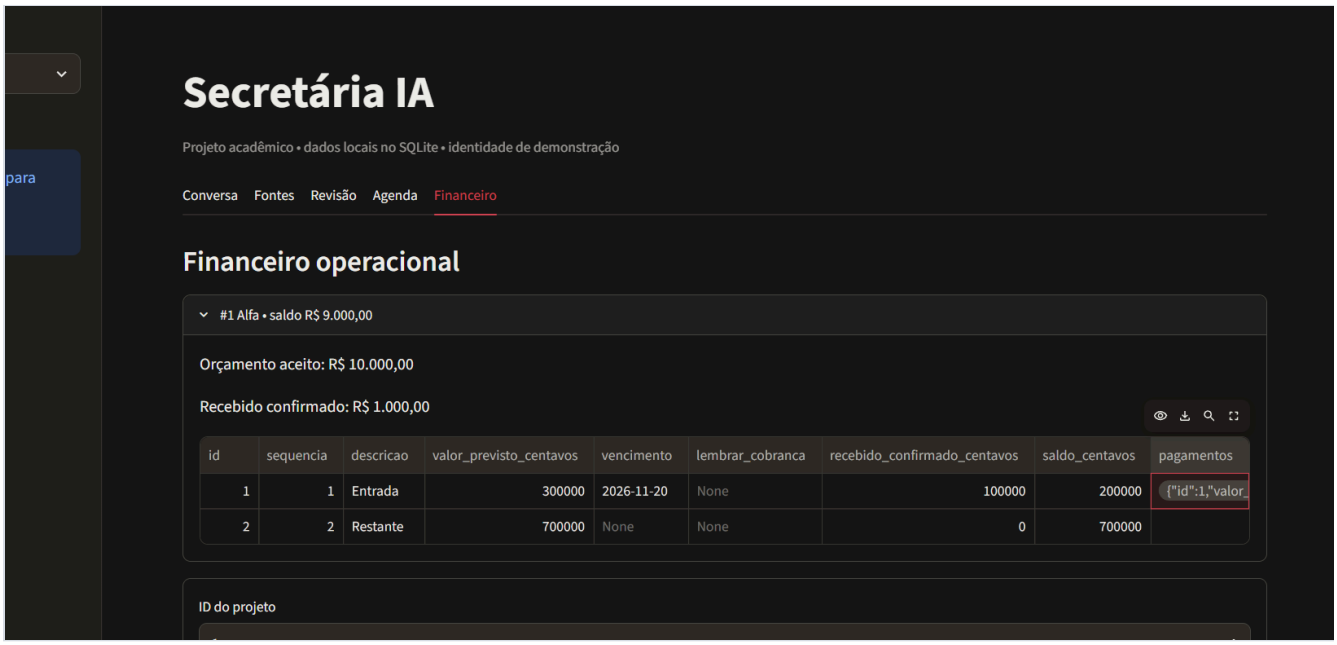


Figura 12 — Financeiro: saldo de R$ 9.000,00 após confirmar o pagamento de R$ 1.000,00.

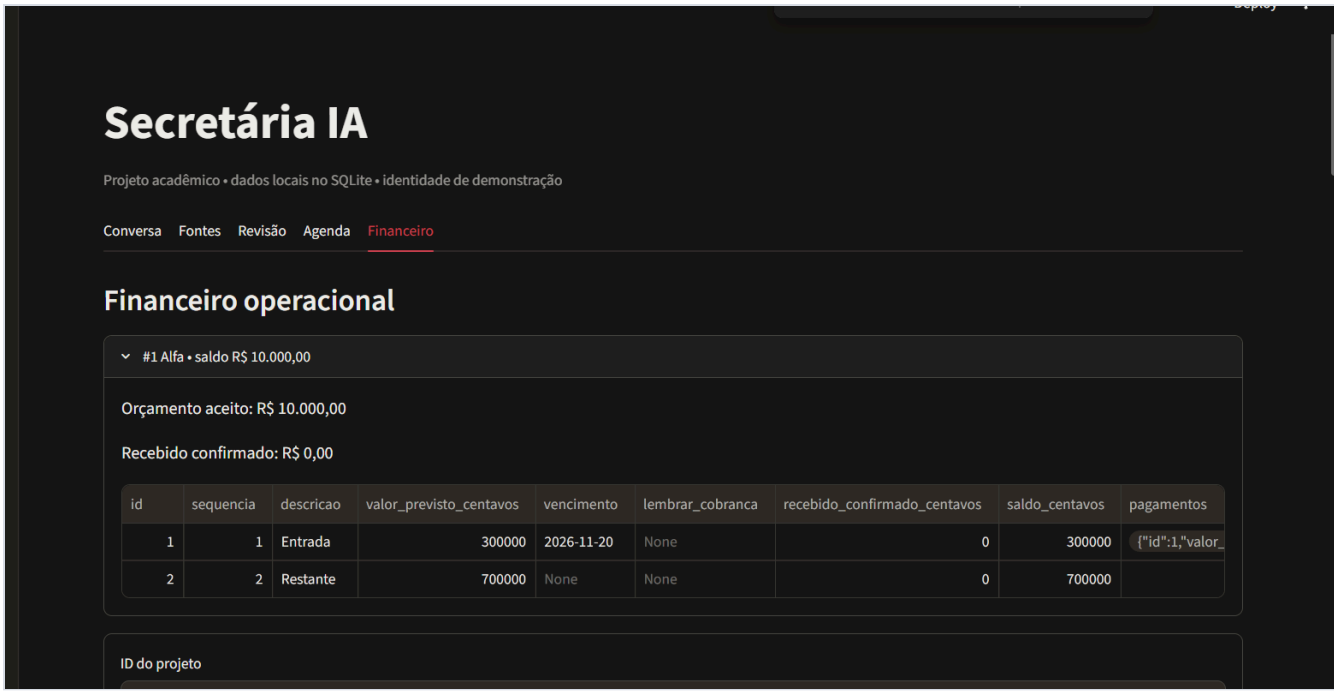


Figura 13 — Estorno: o saldo volta a R$ 10.000,00, e o pagamento original fica preservado na auditoria.



Figura 14 — Permissão: a usuária carla não acessa o financeiro.

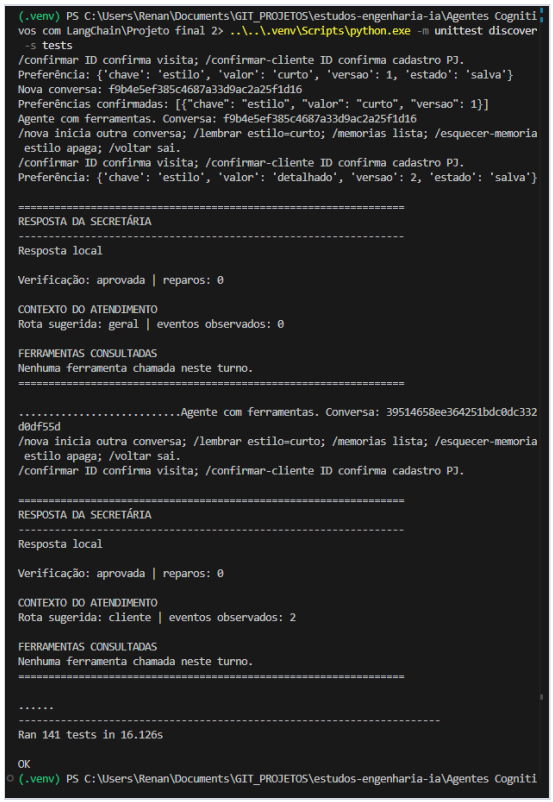


Figura 15 — 141 testes automatizados passando, sem chamar APIs.

8. Limites e melhorias

Limites

- A avaliação tem 4 casos de um turno e não cobre memória multi-turno, propostas nem financeiro. Esses fluxos foram testados manualmente e por testes automatizados. O caso da Figura 10 mostra que o multi-turno pode mudar a escolha de ferramenta.
- Uma verificação "aprovada" não prova que a resposta está certa: o juiz já aprovou uma resposta com um comando inexistente.
- Não há autenticação; os usuários são de demonstração.
- O `stream` da chain não emite tokens.

Melhorias

- Evitar a chamada paralela duplicada (`parallel_tool_calls=False` ou ajuste no prompt) e medir de novo.
- Ampliar o dataset com casos multi-turno, por exemplo uma pergunta de fonte seguida de uma de contagem, e com propostas.
- Fazer o juiz cobrar avisos de negócio, como "orçamento não é recebimento".
- Confirmar por frase no chat, mantendo a gravação em Python, com resumo e uma segunda confirmação.
- Autenticação real antes de qualquer uso com dados reais.

9. Como executar e uso de IA

As instruções estão no `README.md`. Os notebooks e os testes rodam sem chave. O Streamlit (`python -m streamlit run app.py`) e a avaliação precisam de `OPENAI_API_KEY` no `.env`, a partir de `.env.example`. A

instalação foi verificada em um ambiente limpo. A matriz em `docs/matriz_requisitos.md` liga cada item da rubrica (R01–R29) à sua evidência.

Uso de IA no desenvolvimento: assistentes de código (Codex e Claude Code) ajudaram a planejar, escrever código, testes e documentação. As decisões de escopo e de segurança, os testes manuais com a API real e a revisão dos resultados foram feitos por mim, e o histórico está registrado em `docs/uso_de_ia.md` e `docs/execucoes_e_falhas.md`.